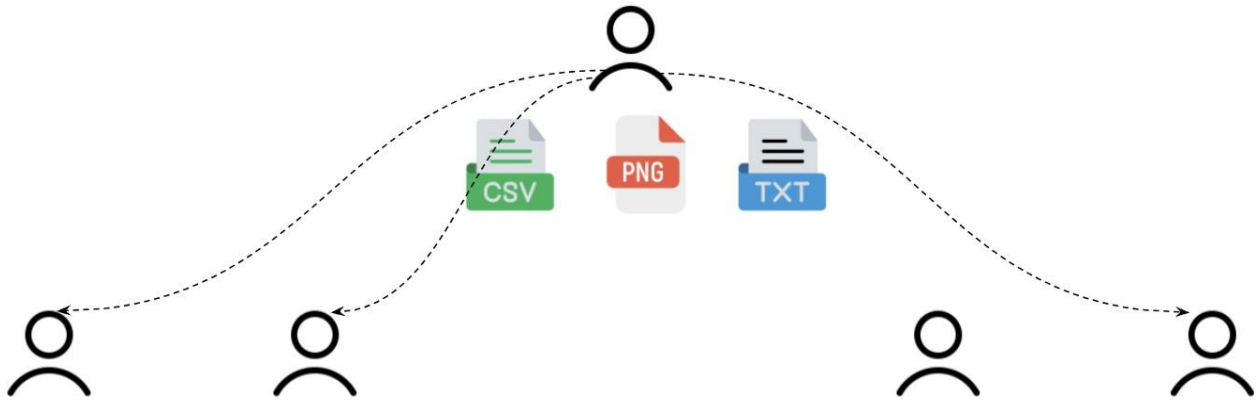
# Assignment 4: Multicast File Sharing

In this assignment, you will design and implement a reliable multicast-based file distribution system for heterogeneous (text and binary) files (e.g. **`train.csv`**, **`test.csv`**, **`descr.txt`** and **`illustration.jpeg`**) in C. The file distribution system has a single sender and can have more than one receiver. The objective is to push the files from the sender to all receivers that are connected to the system. The file distribution system starts a distribution session when the sender starts and ends the session when the sender terminates. The sender is using the multicast protocol to distribute the files to the receivers during the session. The receivers can connect to the distribution system at different times during the session. All receivers who are connected to the distribution system for a sufficient amount of time must get all the files correctly. The overall objective of the file distribution system is to spread the files to the receivers with minimal network load. The network can carry two types of application-level packets created by the file distribution system: data packets and control packets (your design can have other types of packets or you can call them differently). To minimize the network overhead, you broadcast a minimum number of control packets and also a minimum number of data packets. Both types of packets can be retransmitted to handle late arrivals of receivers to a distribution session. If a distribution session is very short lived (i.e., you terminate the sender within a short time of starting it), none of the receivers might have got the files correctly and this behaviour is expected. On the other hand, if a distribution session is long lived all receivers should get all files correctly. The file distribution system should work across multiple machines that are connected by a local area network.



**Core Requirements**

- Enforce Integrity Verification:
  - Chunk-level checksums
  - File-level validation
- Handle late Joiners & Crashes:
  - Receivers joining mid-transmission or restarting must eventually receive all files.
  - Design a mechanism to retransmit missing chunks.
- Starter code usage:
  - Use the provided multicast.h and multicast.c for multicast setup. Example:

```
// Sender initialization
mcast_t *m = multicast_init("239.0.0.1", 5000, 5000);
multicast_send(m, chunk_data, chunk_size);

// Receiver setup
multicast_setup_recv(m);
if (multicast_check_receive(m) > 0) {
    multicast_receive(m, buffer, buffer_size);
}
```

**Tasks**

A: Sender Program **`(sender.c)`**

- Read files from command-line arguments (e.g., `./sender train.csv descr.txt`).
- Chunking:
  - Split files into chunks. Chunk size must be configurable via command-line argument (e.g., ./sender -c 1024 train.csv). Use 1024 bytes as the default if not specified.
  - Add headers with metadata: sequence number, file ID, checksum, total chunks.
- Transmission:
  - Send chunks to the multicast group.
  - Implement reliability (e.g., retransmit chunks cyclically or upon receiver requests).
- Track statistics.

B: Receiver Program **`(receiver.c)`**

- Join the multicast group and listen for chunks.
- Reassembly:
  - Buffer chunks in RAM or write to disk incrementally.
  - Handle out-of-order delivery using sequence numbers.

- Validation:
    - Verify chunk checksums; discard corrupted chunks.
    - Detect and Re-request missing chunks.
- Validate final files.
- Save files to disk (e.g., `train.csv` in `received_files/`).

**Design choices** (Address in Report) try to address at least 2, few of them are:

- Will you interleave chunks or send files sequentially ?
- Impact of different chunk sizes (e.g., 0.5 KB, 1 KB, 4 KB) on network overhead? Metrics:
    - End-to-end Latency: Duration from 1st packet sent until all receivers verify all files.
    - Throughput: Data transfer rate measured in packets per second.
- How will you manage memory if a receiver is processing large files (e.g., buffering chunks in RAM vs. writing to disk immediately) ?
- How will you verify the integrity of received files ?

*Maybe you have better design choices in mind, your design choices here are encouraged!*

**Deliverables**

1. Code i.e. zip as multicast_file_sharing_<group_id>.zip and upload to mycourses, include a README if necessary.
2. Report (2 pages including everything) on design choices and how your solution handles late joiners, crashes, and corruption. Indicate in one paragraph how you worked together (how many meetings) and how each of the team members contributed to the project deliverable. If you do not describe this, you will get 10 penalty points.

**Grading Criteria**

- Multicast heterogeneous files - 40%
- Correctness i.e. files received intact - 40%
- Statistics monitoring of data being transmitted - 10%
- Report justifying design choices - 10%

*This assignment is partially open-ended. While the core requirements (reliability, integrity, late joiner handling) are fixed, you are encouraged to design protocols inspired by those taught in this course or innovate something of your own :) Remember to consider trade-offs and justify your choices in the report.*

Good Luck!